

Laboratorium 5: Kody binarne jednoznacznie dekodowalne

Opracowanie zestawu zadań (część teoretyczna)

Kacper Bednarczuk

Zadanie 1.1: Zastosowanie testu (weryfikacja)

W celu weryfikacji jednoznacznej dekodowalności kodów A, B i C, zastosowano test wiszącego sufiksu (algorytm Sardinas-Pattersona).

Kod A: {1, 01, 001, 000}

- Lista słów kodowych C:** {1, 01, 001, 000}
- Test na prefiksy:** Sprawdzamy, czy którekolwiek słowo kodowe jest prefiksem innego słowa kodowego.
 - 1 nie jest prefiksem 01, 001, 000.
 - 01 nie jest prefiksem 1, 001, 000.
 - 001 nie jest prefiksem 1, 01, 000.
 - 000 nie jest prefiksem 1, 01, 001.
- Lista sufiksów S:** Ponieważ żadne słowo kodowe nie jest prefiksem innego, nie można wygenerować żadnych "wiszących sufiksów". Lista S pozostaje pusta.
- Wniosek:** Test kończy się, ponieważ nie można wygenerować nowych sufiksów.
- Werdykt: Kod A jest jednoznacznie dekodowalny (UD).** Jest to kod prefiksowy, a każdy kod prefiksowy jest z definicji jednoznacznie dekodowalny.

Kod B: {00, 11, 010}

- Lista słów kodowych C:** {00, 11, 010}
- Test na prefiksy:**
 - 00 nie jest prefiksem 11, 010.
 - 11 nie jest prefiksem 00, 010.
 - 010 nie jest prefiksem 00, 11.
- Lista sufiksów S:** Podobnie jak w przypadku Kodu A, żadne słowo kodowe nie jest prefiksem innego. Lista S pozostaje pusta.
- Wniosek:** Test kończy się z pustą listą sufiksów.
- Werdykt: Kod B jest jednoznacznie dekodowalny (UD).** Jest to również kod prefiksowy.

Kod C: {1, 10, 001, 011}

1. **Lista słów kodowych C:** {1, 10, 001, 011}
 2. **Krok 1: Generowanie początkowej listy sufiksów (S1)**
 - Porównujemy pary słów z C:
 - $c_i = 1$, $c_j = 10$. Słowo 1 jest prefiksem 10.
 - Generujemy sufiks: 10 bez 1 $\rightarrow$ 0.
 - **Lista sufiksów S1:** {0}.
 3. **Krok 2: Generowanie kolejnych sufiksów (S2)**
 - Porównujemy sufiksy z S1 ze słowami kodowymi z C:
 - Sufiks $s = 0$, słowo kodowe $c = 001$. Sufiks 0 jest prefiksem 001.
 - Generujemy nowy sufiks: 001 bez 0 $\rightarrow$ 01.
 - Sufiks $s = 0$, słowo kodowe $c = 011$. Sufiks 0 jest prefiksem 011.
 - Generujemy nowy sufiks: 011 bez 0 $\rightarrow$ 11.
 - **Lista sufiksów S2:** {0, 01, 11}.
 4. **Krok 3: Generowanie kolejnych sufiksów (S3)**
 - Porównujemy nowe sufiksy (01, 11) z S2 ze słowami kodowymi z C:
 - Sufiks $s = 01$, słowo kodowe $c = 011$. Sufiks 01 jest prefiksem 011.
 - Generujemy nowy sufiks: 011 bez 01 $\rightarrow$ 1.
 - **Lista sufiksów S3:** {0, 01, 11, 1}.
 5. **Wniosek:** W kroku 3 wygenerowaliśmy sufiks 1, który jest jednocześnie słowem kodowym należącym do zbioru C.
 6. **Werdykt:** Kod C nie jest jednoznacznie dekodowalny (non-UD).
-

Zadanie 1.2: Projektowanie iteracyjne (rozbudowa kodu)

Rozbudowujemy kod $B = \{00, 11, 010\}$ o nowe słowo kodowe.

Kandydat 1: 01

1. **Nowy zbiór kodowy C':** {00, 11, 010, 01}
2. **Test na jednoznaczność dekodowalność:**
 - **Krok 1 (S1):** Porównujemy słowa z C'.
 - $c_i = 01$, $c_j = 010$. Słowo 01 jest prefiksem 010.
 - Generujemy sufiks: 010 bez 01 $\rightarrow$ 0.
 - **Lista sufiksów S1:** {0}.
 - **Krok 2 (S2):** Porównujemy sufiksy z S1 ze słowami z C'.
 - Sufiks $s = 0$, słowo kodowe $c = 00$. Sufiks 0 jest prefiksem 00.
 - Generujemy nowy sufiks: 00 bez 0 $\rightarrow$ 0. Sufiks już istnieje na liście.
 - Sufiks $s = 0$, słowo kodowe $c = 01$. Sufiks 0 jest prefiksem 01.

- Generujemy nowy sufix: 01 bez 0 -> 1 .
 - Sufiks $s = 0$, słowo kodowe $c = 010$. Sufiks 0 jest prefiksem 010 .
 - Generujemy nowy sufix: 010 bez 0 -> 10 .
 - **Lista sufixów S2:** {0, 1, 10} .
 - **Krok 3 (S3):** Porównujemy nowe sufiksy (1 , 10) z C'.
 - Sufiks $s = 1$, słowo kodowe $c = 11$. Sufiks 1 jest prefiksem 11 .
 - Generujemy nowy sufix: 11 bez 1 -> 1 . Sufiks już istnieje.
 - **Lista sufixów S3:** {0, 1, 10} .
 - **Wniosek:** Nie można wygenerować żadnych nowych sufixów. Żaden z wygenerowanych sufixów nie jest słowem kodowym.
3. **Werdykt:** Po dodaniu słowa 01 , **kod pozostał jednoznacznie dekodowalny (UD)**.

Osiągnięcie punktu stałego

W kroku 3 algorytm przestał generować nowe sufiksy. Mimo że proces porównywania mógłby trwać dalej, nie prowadziłby do rozszerzenia listy sufixów. Ponieważ żaden z wygenerowanych sufixów (0 , 1 , 10) nie jest słowem kodowym, test kończy się z wynikiem pozytywnym, a kod jest jednoznacznie dekodowalny.

Kandydat 2: 110

1. **Nowy zbiór kodowy C':** {00, 11, 010, 110}
2. **Test na jednoznaczność dekodowalność:**
 - **Krok 1 (S1):** Porównujemy słowa z C'.
 - $c_i = 11$, $c_j = 110$. Słowo 11 jest prefiksem 110 .
 - Generujemy sufix: 110 bez 11 -> 0 .
 - **Lista sufixów S1:** {0} .
 - **Krok 2 (S2):** Porównujemy sufix 0 z C'.
 - Sufiks $s = 0$, słowo kodowe $c = 00$. Sufiks 0 jest prefiksem 00 .
 - Generujemy nowy sufix: 00 bez 0 -> 0 . Sufiks już istnieje.
 - Sufiks $s = 0$, słowo kodowe $c = 010$. Sufiks 0 jest prefiksem 010 .
 - Generujemy nowy sufix: 010 bez 0 -> 10 .
 - **Lista sufixów S2:** {0, 10} .
 - **Krok 3 (S3):** Porównujemy nowy sufix 10 z C'.
 - Brak słów kodowych zaczynających się od 10 i brak sufixów zaczynających się od słów kodowych.
 - **Wniosek:** Nie można wygenerować nowych sufixów. Żaden z sufixów nie jest słowem kodowym.
3. **Werdykt:** Po dodaniu słowa 110 , **kod pozostał jednoznacznie dekodowalny (UD)**.

Kandydat 3: 011

1. **Nowy zbiór kodowy C'**: {00, 11, 010, 011}
2. **Test na jednoznaczność dekodowalność:**
 - **Krok 1 (S1):** Porównujemy słowa z C'.
 - Brak słów kodowych będących prefiksami innych.
 - **Lista sufiksów S1:** {} (pusta).
 - **Wniosek:** Test kończy się natychmiast, ponieważ nie można wygenerować żadnych sufiksów.
3. **Werdykt:** Po dodaniu słowa 011, **kod pozostał jednoznacznie dekodowalny (UD)**. Jest to kod prefiksowy.

Podsumowanie: Wszyscy trzej kandydaci (01, 110, 011) mogą zostać użyci do poprawnego rozszerzenia kodu B, ponieważ w każdym przypadku kod wynikowy jest jednoznacznie dekodowalny.

Zadanie 1.3: Oszacowanie efektywności

Wybieramy kod utworzony z **Kandydatem 3**.

Wybrany kod 4-symbolowy: $C = \{00, 11, 010, 011\}$

Mapowanie i długości:

- $s_1 \rightarrow 00$ (długość $d_1 = 2$), $P(s_1) = 0.5$
- $s_2 \rightarrow 11$ (długość $d_2 = 2$), $P(s_2) = 0.2$
- $s_3 \rightarrow 010$ (długość $d_3 = 3$), $P(s_3) = 0.2$
- $s_4 \rightarrow 011$ (długość $d_4 = 3$), $P(s_4) = 0.1$

1. Obliczenie średniej długości słowa kodowego (D)

$$D = P(s_1) \cdot d_1 + P(s_2) \cdot d_2 + P(s_3) \cdot d_3 + P(s_4) \cdot d_4$$

$$D = (0.5 * 2) + (0.2 * 2) + (0.2 * 3) + (0.1 * 3)$$

$$D = 1.0 + 0.4 + 0.6 + 0.3$$

$$D = 2.3 \text{ bity na symbol.}$$

2. Porównanie z kodem stałorozmiarowym

Do zakodowania 4 unikalnych symboli potrzebujemy $\log_2(4) = 2$ bity na symbol w kodzie stałorozmiarowym. Przykładowy kod stałorozmiarowy to {00, 01, 10, 11}. Jego średnia długość wynosi zawsze 2 bity na symbol.

W naszym przypadku średnia długość (2.3 bity) jest nieco wyższa niż w optymalnym kodzie stałorozmiarowym. Dzieje się tak, ponieważ nasz kod nie jest optymalny dla podanego rozkładu prawdopodobieństw (np. najczęstszy symbol s_1 ma taką samą długość jak rzadszy s_2).

3. Porównanie z kodowaniem ASCII

Standardowe kodowanie ASCII używa 8 bitów na symbol.

- **Nasz kod:** 2.3 bita/symbol
- **ASCII:** 8 bitów/symbol

Kompresja (redukcja rozmiaru) w stosunku do ASCII:

$$\text{Kompresja} = (1 - (\text{Długość_naszego_kodu} / \text{Długość_kodu_ASCII})) * 100\%$$

$$\text{Kompresja} = (1 - (2.3 / 8)) * 100\%$$

$$\text{Kompresja} = (1 - 0.2875) * 100\%$$

$$\text{Kompresja} = 0.7125 * 100\% = 71.25\%$$

Wniosek: Nasz kod zmiennorozmiarowy oferuje **71.25%** kompresji w stosunku do standardowego kodowania ASCII dla źródła o podanej charakterystyce. Oznacza to, że plik zakodowany naszym kodem byłby o 71.25% mniejszy niż ten sam plik w kodowaniu ASCII.